

**Document number:** P1832R0

**Date:** 2019-08-05

**Reply-to:** John McFarlane, [wg21@john.mcfarlane.name](mailto:wg21@john.mcfarlane.name)

**Audience:** SG6, SG14, SG15, SG19

# Improving Debug Builds Inline With User Expectation

## Abstract

Some users complain that modern C++ abstractions make it hard to debug their programs. This document explores one of the reasons for these complaints and suggests a possible solution for implementers.

## Introduction

This document is written in response to the assertion that "non-optimized build performance is important" as stated in the article, "["Modern" C++ Lamentations](#)". The article, written by a game developer, sparked discussion on [social media](#) and the [SG14 reflector](#) which did not conclude with concrete action to be taken.

More recently, SG6 and SG14 reviewed linear algebra papers in Cologne and game development was identified as a target audience for these library components. I opined that without any further action to address the assertion, those components — and an increasing proportion of the standard library as a whole — would be rejected by a growing number of C++ users.

It is my view that the assertion forewarns of a scalability problem that C++ must address in order to continue on its current course.

## Background

### Debug Builds

During the development cycle, it is common for C++ users to produce binaries that are optimised for testing and debugging. These are often called *debug builds* — to be contrasted with *release builds* which are optimised for production use.

It is accepted that debug builds are bigger and execute more slowly than release builds of the same program. They may include correctness checks, and symbol and debugging information. They may also disable optimizations which hamper the compiler's ability to provide helpful debugging information.

# Function Inlining

A very common optimization which causes the loss of debugging information is *function inlining*. Function inlining effectively removes functions and, in doing so, removes or obscures debugging information associated with those functions.

The lack of function inlining is one reason why a debug build may be bigger and execute more slowly.

## Library-Based Abstractions

Unfortunately, a growing number of library-based abstractions rely on function inlining in order to maintain zero run-time overhead. Many libraries are affected, including the standard library. The abstractions are characterised by deep call trees which collapse down to efficient machine code.

## The Dilemma

Without function inlining, these abstractions incur significant cost in terms of binary size and run-time performance. With function inlining, the same abstractions obfuscate stack traces and otherwise hamper the ability of tools to provide the user with helpful debugging information.

This causes a dilemma which is felt most acutely in abstractions that have the highest ratio of compile-time complexity to run-time code generation.

For example, many numerics libraries offer abstractions over fundamental arithmetic types. A substantial amount of source code will routinely result in [a single arithmetic instruction](#). However, the problem [ranges](#) far beyond purely arithmetic abstractions.

## Problem Analysis

### One User's Solution is Another User's Problem

One way in which library designers attempt to address concerns over performance of debug builds is to try and single out their functions for inlining. This fails for two reasons.

Firstly, explicit inlining as an optimization hint is a mistake. A modern tool chain does a far better job of determining if a function is a good candidate for inlining. Many factors contribute to this determination including: target platform, caller, build configuration and profile guidance. Assuming the user made the right choice on a given day (unlikely), the same choice may be wrong tomorrow.

Secondly, the intent to inline is based on ownership of the function. It's likely that *users* of the library DO wish for inlining of its functions in debug builds. Conversely, *authors* of the library DO NOT wish for inlining of the very same functions. Hence, no annotation or keyword could possibly cover both use cases.

In short, explicit inlining is a bad tool for the wrong job.

## One of Several Problems for C++

A common feature of modern C++ functions is that they are delivered via header files. Comparable traditional C++ and C functions would be defined in source files, making it easy for the library author or library user to optimize them in isolation.

This also makes it easy for the library user to step over them during an interactive debugging session. Indeed, if the user is unsure whether the function is their own, they may habitually chose the 'step into' facility of the debugger and automatically traverse the subsection of the call graph that is under their control.

This paper focuses on run-time performance of debug builds and not the quality of interactive debugging experience. However, it is a closely-related topic and this proposal hopes to improve it as a side effect. It has also been tackled by [IDEs](#) and [GDB extensions](#) suggesting that is a problem worthy of attention in its own right.

## A Niche Problem

The problem of debug build performance has a non-technical dimension.

Firstly, it affects programs where performance of debug builds is critical. This excludes batch programs and event-based user interfaces where slower response to user input is not a deal-breaker.

Secondly, it only affects developers who use debuggers. There is [disagreement](#) about whether interactive debugging is even necessary in the era of test-driven development. But it is clear that it is deemed necessary by users in specific domains and has been for a long time.

It is plausible that developers who suffer most from debug build performance are not well represented within WG21. To them, the committee appears unsympathetic.

## Somebody Else's Problem

Where the standard adds features which cause pain to users, it adds insult to injury by describing those features as a 'quality of implementation' issue. That description is both accurate and inadequate. Not only does the committee have a responsibility to deal with the consequences of its choices. But it also has a unique ability to mediate between users and implementors. Hopeful this is a role that SG15 can fulfil.

## Conclusion of Problem Analysis

The course that C++ has been following for a long time takes it in a direction which is causing some users an increasing number of productivity problems including:

1. slow, bloated debug builds;
2. back traces that include unwanted noise from extra-project code;
3. hard-to-step-through call graphs; and
4. slow build times.

Function inlining mitigates 1), 2) and 3) but can also adversely affect 1), 3) and 4). Thus the user faces a dilemma for which we offer no satisfactory solution. The author suggests that — aside from 4) which is an pressing issue in its own right — it is the dilemma which deserves attention and not individual problems, 1), 2) and 3).

## Straw Man Solution

In order to stimulate discussion around a workable solution to the dilemma, the following sub-section hypothesises a change to popular tool chains which — if implementable — would provide an implementation-specific solution that could apply to existing source code unchanged.

The solution works by considering pre-existing information about the origin of a function definition and using it to solve the dilemma of whether or not to inline that function.

## Proposed Change to GCC Option, `-Og`

[gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html](http://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html)

`-Og`

Optimize debugging experience. `-Og` should be the optimization level of choice for the standard edit-compile-debug cycle, offering a reasonable level of optimization while maintaining fast compilation and a good debugging experience. It is a better choice than `-O0` for producing debuggable code because some compiler passes that collect debug information are disabled at `-O0`.

Like `-O0`, `-Og` completely disables a number of optimization passes so that individual options controlling them have no effect. Otherwise `-Og` enables all `-O1` optimization flags except for those that may interfere with debugging:

```
-fbranch-count-reg -fdelayed-branch  
-fif-conversion -fif-conversion2  
-finline-functions-called-once
```

```
-fmove-loop-invariants -fssa-phiopt  
-ftree-bit-ccp -ftree-pta -ftree-sra
```

Additionally, `-Og` enables the following optimization flags for functions defined in system headers (i.e. defined in headers found via `-isystem`):

```
-finline-functions-called-once  
-finline-functions  
-finline-small-functions
```

## MSVC Option, `/Ob`

A similar MSVC facility can be envisaged with the experimental `/external:I` switch as a substitute for `-isystem`. This would involve a combination of switches such as

```
cl.exe /Od /Ob3 /external:IC:\somebody\elses\headers  
/IC:\my\project\headers C:\my\project\source\file.cpp
```

where `/Ob3` is a novel function inlining option which discriminates between 'system' (i.e. dependency) headers and 'project' headers.

## FAQ

Q: How does the proposed tool chain enhancement help?

A: It helps by drawing a distinction between the user's functions and the functions of a user's dependencies. It is assumed that the user only wishes to debug their own code and would rather optimise dependency code. Dependency code should already be included in the translation unit via a different header search path option. The compiler can use that change to make the choice to inline.

Q: Does this proposal work?

A: I have absolutely no idea. Optimising compilers are incredibly sophisticated and complex tools which I haven't devoted the time to understand. It is entirely possible, for example, that the architecture of a particular compiler makes it especially difficult to selectively enable and disable inlining on a per-function basis. But even if this technique worked half of the time, I posit that it would improve the situation for users.

Q: Why inlining, specifically, and not other optimizations which would make dependency code faster?

A: For the sake of simplicity, this paper concentrates on what the author believes is the single most significant optimization technique.

# Impact On The Standard

None.

# Acknowledgements

Thanks to Matthias Kretz for providing a valuable perspective as the author of a numerics library.